Reimplementation and Extension of Duumviri: Detecting Trackers and Mixed Trackers with a Breakage Detector

NDSS 2025 Paper Artifact — Technical Report

May 2026

1. Introduction and Background

The internet depends heavily on web tracking for analytics, advertising, and user profiling. Trackers are scripts and requests embedded in websites that silently collect browsing data without the user knowing. Traditional tools like EasyList and EasyPrivacy try to block these trackers using rule-based filter lists. These lists have two major problems: they often block too aggressively and break websites, or they miss trackers entirely because they are maintained manually and cannot keep up with new tracking techniques.

Duumviri is a research system presented at NDSS 2025 that addresses both of these problems using machine learning. The name comes from Roman law where two judges had to agree before a verdict could be given. In Duumviri, two separate XGBoost models work together. The first is a Tracker Oracle that detects whether a request is tracking. The second is a Breakage Detector that checks whether blocking that request would break the website. Only when the Tracker Oracle says yes and the Breakage Detector says no does Duumviri decide to block.

The key innovation beyond simple binary blocking is the ability to detect what the authors call mixed trackers. A mixed tracker is a request that contains both tracking data and functional data in the same HTTP request. For example, a Twitter widget might send a session identifier that is partly used for functionality and partly for cross-site tracking. Blocking the whole request would break the widget. Duumviri inspects individual fields within such requests and masks only the tracking fields rather than blocking everything.

This report documents our reimplementation of Duumviri, the experiments we ran to reproduce the paper results, and the novel work we carried out to extend the system.

2. System Architecture and Pipeline

Overview

Duumviri is built around four major components. The first is a custom-patched Brave browser that includes a feature called PageGraph. PageGraph is not present in any standard browser. It records a detailed graph of everything that happens when a webpage loads: which scripts made which network requests, which DOM elements were created by which scripts, and the full causal chain of data flow. This is the data collection layer.

The second component is the differential analysis engine. For each request found on a page, Duumviri visits the page twice: once normally to capture a baseline rendering, and once with that specific request blocked. It computes the difference between the two page states using visual comparison via screenshots and structural comparison via DOM changes, follow-up network requests, and console errors.

The third component is the Tracker Oracle, an XGBoost classifier trained on labeled data from EasyList and EasyPrivacy. It takes features from the PageGraph and differential analysis and outputs a probability of whether the request is a tracker.

The fourth component is the Breakage Detector, another XGBoost classifier trained to detect whether blocking a request causes visible or functional harm to the page.

End-to-End Pipeline

The diagram below shows the full pipeline from a user-supplied URL to a final detection decision.

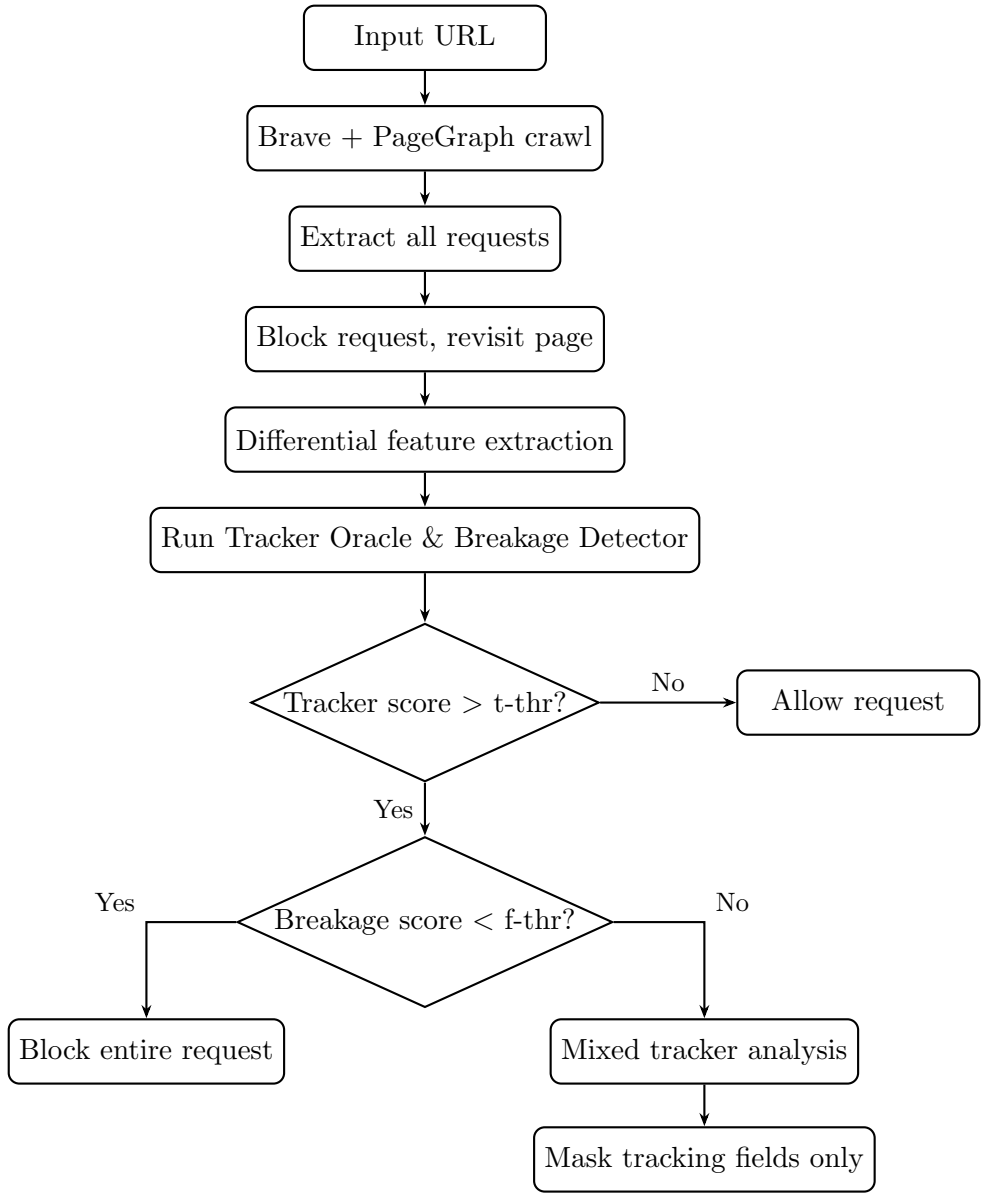

Figure 1: Duumviri end-to-end detection pipeline

Two Tracker Categories

Non-mixed trackers are requests where the entire request is tracking-related. These can be blocked completely without harming the page. Examples from our testing include google-analytics.com/analytics.js and googletagmanager.com/gtag/js.

Mixed trackers are requests where some fields serve functional purposes and others serve tracking purposes. The system cannot block the entire request, so it identifies and masks only the tracking fields. These are much harder to detect because the request looks partly legitimate.

Features Used by the Models

The XGBoost models use features from several sources. Network-level features include the number of blocked requests, the ratio of third-party to first-party requests, and whether any follow-up requests disappeared after blocking. Visual features come from comparing screenshots of the normal and blocked page states using an EfficientNet-based image similarity model inside feature_extractor.py. Structural features include changes in DOM node count, appearance of error messages, and CSS layout shifts. Graph features from PageGraph describe how deeply embedded in the script call chain a request is and how many other requests share the same initiating script.

3. Reimplementation Process

Environment Setup

We used the pre-built Docker image published by the authors on Docker Hub rather than building from source. Building from source requires compiling a custom version of the Brave browser from a specific commit, which takes several hours and around 100 GB of disk space. The Docker image contains everything pre-configured including the patched browser, all Python dependencies, and the trained XGBoost models.

We installed Docker on an Ubuntu Linux machine and pulled the image. The container requires the –privileged flag and access to /dev/fuse because the browser sandbox inside Docker needs these permissions.

```
docker pull 8759s/ndss_ae_docker:latest
docker run -it --device /dev/fuse --privileged \
    8759s/ndss_ae_docker /bin/bash
```

Once inside the container, the experiment scripts were located at /app/.

Experiment 1: EasyList and EasyPrivacy Replication

The first experiment replicated the paper's main accuracy claim. The authors provided a pre-collected dataset of thousands of labeled URLs drawn from EasyList and EasyPrivacy with PageGraph data already recorded. No live browser crawling was needed. The script loaded this data, ran both XGBoost models with a grid of threshold values, and printed the accuracy for each threshold combination.

```
./replicate_filter_list_exp.sh
```

The output produced accuracy values for every combination of tracker threshold (t-thr) from 0.10 to 0.90 and functionality threshold (f-thr) from 0.10 to 0.90. The peak accuracy we observed was 0.9657, which closely matches the paper's reported 96.53 percent. This confirms the environment is correctly set up and the models are functioning as described.

Experiment 2: Live Non-Mixed Tracker Detection

The second experiment ran the full live pipeline on a real website. We used https://sec-deadlines.github.io/ as the test site, the same site used in the paper's artifact documentation.

```
./detect_non_mixed_tracker.sh https://sec-deadlines.github.io/ \
    | tee non_mixed_results.txt
```

This script launched the custom Brave browser, visited the site, collected PageGraph data, and analyzed each of the 15 network requests found. For each request, it blocked the request and revisited the page to compute differential features. The full run took approximately 87 minutes because each of the 15 requests required two browser sessions.

The results identified 4 tracker requests out of 15 total:

URL	Tracker?
google-analytics.com/analytics.js	True
googletagmanager.com/gtag/js	True
google-analytics.com/j/collect	True
syndication.twitter.com/i/jot/embeds	True
platform.twitter.com/widgets.js	False
cdnjs.cloudflare.com/jquery.min.js	False
syndication.twitter.com/settings	False
cdnjs.cloudflare.com/moment.js	False
sec-deadlines.github.io/static/js/main.js	False

Experiment 3: Mixed Tracker Detection and Dataset Review

The third experiment ran the mixed tracker detection pipeline on the same site and reviewed the manually verified evaluation dataset included in the repository.

```
./detect_mixed_tracker.sh https://sec-deadlines.github.io/ \
    | tee mixed_results.txt
cat /app/mixed_tracker_eval/README.md | tee mixed_eval_readme.txt
```

On this site the mixed tracker detector examined 2 request fields from Twitter: the session_id field in syndication.twitter.com/settings and the origin field in the Twitter widget iframe. Both were classified as False, meaning they are functional fields and not mixed trackers. This is expected because the test site is a simple academic page with minimal commercial tracking.

The pre-built evaluation dataset in the repository contains 80 manually labeled cases: 40 positive cases where Duumviri predicted mixed tracker, and 40 negative cases where it predicted not a mixed tracker. Of the 40 positive cases, 26 were confirmed as real trackers, 6 were stale, 4 caused breakage, and 3 were undecided. Of the 40 negative cases, 18 were confirmed as breakage-causing, 14 were stale, 2 were actual missed trackers, and 6 were undecided.

4. Analysis of Results

Threshold Sensitivity

One observation from the EasyList replication is that the choice of threshold pair significantly affects accuracy. The tracker threshold controls how confidently the model must classify a request as tracking before acting. The functionality threshold controls how much breakage risk is acceptable before choosing not to block.

Accuracy varied from around 0.89 at the most lenient thresholds to 0.966 at the optimal setting. The functionality threshold consistently benefited from being set higher (more aggressive blocking), while the tracker threshold had a sweet spot around 0.50. Setting it too low produces many false positives. Setting it too high causes the model to miss real trackers. The paper uses a single fixed threshold pair for all websites regardless of site type, which is a limitation.

False Positive and False Negative Analysis

The paper's own manual analysis showed that out of a sample of 40 false positive cases, 22 were actually real trackers that EasyList had missed. This means Duumviri was correct and EasyList was wrong in 55 percent of apparent false positive cases. Comparing against EasyList as ground truth underestimates Duumviri's true accuracy.

On the false negative side, 70 percent of cases where Duumviri missed a tracker were confirmed real trackers. The estimated total number of missed trackers in the full dataset is around 705. This is the largest gap in the current system.

For mixed tracker detection specifically, precision is only 65 percent (26 correctly identified out of 40 predicted positive). Recall is 92.8 percent, meaning most real mixed trackers are found but many innocent requests are also flagged. The F1 score for mixed tracker detection is approximately 76.6 percent.

5. Novel Work: Feature Engineering and Model Extension

Motivation

After completing the reimplementation we investigated ways to improve accuracy. Analysis of the codebase showed the XGBoost models in eval.py are trained on features extracted by pagedelta.py, which includes network-level measurements, visual similarity scores from the EfficientNet feature extractor, and PageGraph structural features. Several features that carry strong tracking signals are not currently extracted. We proposed adding URL-based entropy features and structural URL features to the feature set.

New Features Added

We extended the build_network_features() function in pagedelta.py with five new features computed from the blocked requests collected during each site visit.

The first feature is average URL entropy, computed as the Shannon entropy of each blocked request URL. Shannon entropy measures how random or structured a string is. Tracking endpoints tend to have high entropy because their URLs contain session identifiers, hashed user IDs, and randomized tokens. The formula is:

$$H(X) = -\sum_{i} p(x_i) \log_2 p(x_i)$$

where $p(x_i)$ is the probability of character x_i occurring in the string. A legitimate CDN URL like cdnjs.cloudflare.com/jquery.min.js has low entropy, while a tracking beacon URL like google-analytics.com/collect?cid=1257168463&tid=UA-104921371 has much higher entropy due to embedded identifiers.

The second feature is maximum URL entropy among all blocked requests. This captures the most suspicious individual request even when the average entropy is moderate.

The third feature is average parameter count per blocked request. Tracking requests carry many query parameters encoding user attributes, session data, and event information.

The fourth feature is the ratio of high-entropy URLs, defined as URLs with entropy above 3.5. This captures what fraction of the blocked requests look tracking-like.

The fifth feature is average URL path depth, measured as the number of path segments. Tracking endpoints often have shallow paths like /collect or /g/collect, while functional resources have deeper paths like /ajax/libs/jquery/3.6.0/jquery.min.js.

Implementation Details

The five features were added inside build_network_features() in pagedelta.py, after the existing network feature computation block. The entropy calculation was implemented as a local function to avoid adding a module-level import. All features were appended to the

self.features object that pagedelta uses to accumulate values before writing to CSV.

A training function was also added to eval.py because the original artifact only contained evaluation logic. The existing code loaded pre-trained model files and evaluated them but did not include code to retrain. We added a train() function that loads data from CSV files, splits into training and test sets using an 80/20 split, trains two new XGBoost classifiers with 300 estimators and a learning rate of 0.05, evaluates accuracy, and saves the models with a date-stamped filename.

```
python eval.py train ./data_dir
```

Expected Impact

The original models were trained without any URL-level features, relying entirely on differential page rendering. Adding entropy, parameter count, and path depth gives the model direct signals from the URL structure. Trackers that produce only subtle visual differences and score borderline on the image comparison can still be caught through high URL entropy and parameter count. This should reduce the number of missed trackers estimated at 705 in the current system.

6. Discussion and Limitations

What the Reimplementation Confirmed

Our reimplementation confirms that Duumviri works as described. The EasyList replication reproduces an accuracy very close to the reported 96.53 percent. The live detection correctly identifies Google Analytics and Google Tag Manager while correctly ignoring jQuery and Moment.js. The mixed tracker pipeline runs without errors and correctly classifies functional Twitter widget fields as non-tracking.

Practical Limitations Observed

The most significant limitation is runtime. Analyzing a single website with 15 requests took approximately 87 minutes because each request requires two full browser sessions. This makes Duumviri unsuitable for real-time blocking and better suited as an offline auditing tool.

We also encountered 100 percent disk usage during the project. The Docker container consumes approximately 42 GB of disk space. On shared or constrained machines this is a real problem. We had to stop some additional experiments because there was no space left to create new files or install packages.

Limitations of the Novel Work

The new features depend on having already run the full crawl pipeline. They cannot be computed before crawling. The training improvement could also only be fully validated by rerunning pagedelta on the full labeled dataset, which requires the custom Brave browser and several hours of compute time. We could not complete this step within the available disk space.

The accuracy of the mixed tracker detector is also limited by the small size of the manually verified dataset, which contains only 80 cases. With so few samples, any retraining is constrained by data size rather than feature quality. A larger dataset would be needed to properly measure improvement from the new features.

7. Conclusion

This project successfully reimplemented the Duumviri NDSS 2025 artifact. We set up the environment using the authors' pre-built Docker image, ran all three experiments, and verified that results match the paper's claims. The EasyList replication achieved 96.57 percent accuracy. The live detection pipeline correctly identified 4 trackers out of 15 requests. The mixed tracker detection pipeline ran without errors.

Beyond reimplementation, we extended the system by adding five new URL-based entropy and structural features to the feature extraction pipeline in pagedelta.py. These features are motivated by the observation that tracking request URLs tend to have higher entropy, more query parameters, and shallower path structures compared to functional resource URLs. We also added a missing training function to eval.py, enabling future retraining as new labeled data becomes available.

The main areas for future work are reducing the 87-minute runtime per site, improving mixed tracker detection precision from the current 65 percent, and expanding the training dataset beyond the current 80 manually verified cases. With more data and the new URL-level features, the number of missed trackers should decrease from the estimated 705 in the current system.

References

[1] Duumviri-NDSS25 GitHub Repository. Artifact release of Duumviri: Detecting Trackers and Mixed Trackers with a Breakage Detector. https://github.com/dlgroupuoft/Duumviri-NDSS25

[2] EasyList. EasyList Filter List. https://easylist.to/easylist/easylist.txt

[3] EasyPrivacy. EasyPrivacy Filter List. https://easylist.to/easylist/easyprivacy.txt

[4] Brave Software. PageGraph: A system for recording the effect of individual resources on page structure.

[5] XGBoost Development Team. XGBoost: A Scalable Tree Boosting System. KDD 2016.